

An Agentic Intelligent Code Refactoring Assistant for Legacy Systems

Project ID

R26-SE-008

Project Proposal Report

Nanayakkara G.L.C.S – IT22103918

B.Sc. (Hons) Degree in Information Technology Specialized in
Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Nanayakkara G.L.C.S	IT22103918	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Dr. Kalpani Manathunga	17/03/2020	
Mr. Jeewaka Perera	13-03-2020	

ABSTRACT

Legacy software maintenance has become increasingly challenging with the rapid adoption of AI-assisted development tools such as GitHub Copilot and large language models. While these tools significantly improve developer productivity, they can also contribute to technical debt when generated code is integrated without proper architectural planning or systematic refactoring. As a result, software systems often accumulate large and complex code structures, leading to code smells, reduced maintainability, increased coupling, and long-term scalability issues. These challenges create significant difficulties for developers who must understand, maintain, and evolve such systems over time.

Although automated refactoring tools exist, most current solutions provide isolated refactoring recommendations without considering global architectural context, dependencies between refactoring operations, or strategic planning of transformations. Consequently, applying multiple refactorings sequentially may lead to conflicts, suboptimal restructuring, or limited improvement in software quality.

This research proposes a Refactoring Decision and Planning Agent (RDP Agent) as the strategic planning component of a multi-agent automated refactoring framework. The proposed agent analyzes structured quality reports generated by a code understanding module and identifies potential refactoring opportunities. A hybrid decision-making mechanism combining rule-based reasoning, machine learning-based suitability prediction, and Multi-Criteria Decision Analysis (MCDA) is used to evaluate alternative refactoring strategies. The system considers factors such as implementation complexity, behavioral risk, dependency relationships, and expected quality improvement to generate a structured and dependency-aware refactoring plan.

The proposed system will be implemented using Python and integrated with static code analysis techniques based on Abstract Syntax Tree (AST) parsing. Experimental evaluation will be conducted using selected open-source software repositories to assess the effectiveness of the planning agent in generating executable and quality-improving refactoring plans. The expected outcome is an intelligent planning mechanism that improves automated refactoring workflows, reduces technical debt, and supports sustainable AI-assisted software development.

Keywords: Automated Refactoring, Refactoring Planning, Technical Debt, Machine Learning, Multi-Agent Systems, Software Maintenance

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor , co-supervisor , external supervisor and all the contributors to this research for their invaluable support and guidance throughout the process. I extend my deepest appreciation to all those who contributed, directly or indirectly, to the development of this work.

A special thank you goes to Dr Kalpani Manathunga and Mr Jeewaka Perera whose expertise and continuous encouragement were crucial to this project. I also wish to acknowledge the support and resources provided by the Department of Software Engineering at the Sri Lanka Institute of Information Technology.

Furthermore, I would like to express my gratitude to the contributors of open-source refactoring tools such as SonarQube, CodART, RefactorGPT, and RefAgent, which greatly inspired and influenced this work. Their frameworks and research provided a solid foundation for this research, and their contributions to the field of automated software refactoring have been indispensable.

Lastly, I would like to thank my peers and colleagues for their insightful feedback, constructive criticism, and moral support.

TABLE OF CONTENT

Contents

DECLARATION.....	2
ABSTRACT.....	3
ACKNOWLEDGEMENT.....	4
TABLE OF CONTENT	5
LIST OF TABLES.....	6
LIST OF FIGURES	7
LIST OF APPENDICES.....	9
1. INTRODUCTION	10
1.1 Background and Context.....	10
1.2 Literature Review.....	11
1.3 Research Gap	15
2. OBJECTIVE	17
2.1 Main Objective.....	17
2.2 Specific Objectives	17
3. METHODOLOGY	18
4. HIGH-LEVEL SYSTEM ARCHITECTURE.....	19
7. BUDGET AND JUSTIFICATION	26
8. WORK BREAKDOWN STRUCTURE (WBS).....	28
9. GANTT CHART.....	29
10. REFERENCES LIST	31
11. APPENDICES.....	32

LIST OF TABLES

Table 1 Summary of Related Work in Automated Software Refactoring.....	12
Table 2 Functional Requirements of the Refactoring Decision and Planning Agent	24
Table 3 Non-Functional Requirements of the Refactoring Decision and Planning Agent	24
Table 4 Estimated Budget for System Implementation and Experimentation.....	26
Table 5 Task Distribution Among Team Members.....	28
Table 6 Project Timeline Alignment	28
Table 7 Gantt chart.....	29

LIST OF FIGURES

Figure 1 High-Level Architecture of the Automated Multi-Agent Refactoring System.....	20
Figure 2 Internal Architecture of the Refactoring Decision and Planning Agent	22
Figure 3 Workflow of the Refactoring Planning Process.....	22

LIST OF ABBREVIATIONS

RDP – Refactoring Decision Planning

AST – Abstract Syntax Tree

AI – Artificial Intelligence

LLM – Large Language Model

MCDA – Multi-Criteria Decision Analysis

DSR – Design Science Research

IDE – Integrated Development Environment

IEEE – Institute of Electrical and Electronics Engineers

CISQ – Consortium for Information & Software Quality

PMD – Programming Mistake Detector (Static Code Analysis Tool)

NSGA – Non-dominated Sorting Genetic Algorithm

SOLID – Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion Principles

SDG – Sustainable Development Goals

UI – User Interface

WBS – Work Breakdown Structure

IT – Information Technology

LIST OF APPENDICES

Appendix A Detailed Multi-Agent Refactoring System Architecture	32
Appendix B Internal Architecture of the Refactoring Decision & Planning Agent.....	33
Appendix C Refactoring Decision Algorithm.....	33
Appendix D Example Code Quality Report (Input)	34
Appendix E Example Refactoring Plan Output	34
Appendix F MCDA Decision Matrix.....	35

1. INTRODUCTION

1.1 Background and Context

The software engineering landscape is rapidly changing with the widespread adoption of AI-assisted development tools such as GitHub Copilot and large language models. Many developers now rely heavily on these tools to generate code quickly using natural language prompts. While this approach significantly improves development speed, it often leads to poor architectural planning and unstructured code integration.

In many real-world projects, AI-generated code is directly copied and pasted into existing systems without proper modularization or design review. As a result, single files may grow to thousands of lines of code, creating “God classes,” long methods, high cyclomatic complexity, tight coupling, and violations of SOLID design principles. These issues increase technical debt, reduce maintainability, introduce security vulnerabilities, and negatively affect system scalability and performance.

Over time, such systems become difficult to understand and maintain, especially when new developers join the project. This problem affects multiple stakeholders in the software development lifecycle:

- Interns and Junior Developers struggle to understand large, unstructured codebases, leading to stress and slow onboarding.
- Senior Developers spend significant time reviewing, debugging, and refactoring AI-generated code instead of focusing on innovation.
- Organizations and Project Owners face increased maintenance costs, slower feature development, and higher risks of system failure.

Improving readability, maintainability, performance, and security in legacy and AI-assisted software systems is essential for long-term software sustainability. Recent industry evidence highlights both the scale and urgency of this challenge:

Technical debt at a trillion-dollar scale: Poor software quality creates massive long-term cost. The IT-CISQ report estimates that accumulated software technical debt in the United States reached approximately \$1.52 trillion in 2022 [1].

The problem is growing as AI tools become increasingly common in software development. Studies show that more than 80% of developers are already using or planning to use AI-assisted tools to support coding, debugging, and development tasks. Major technology companies have also integrated AI into their development workflows, highlighting how AI-generated or AI-assisted code is becoming a standard practice in modern software engineering[2].

Large technology companies have publicly reported the growing role of AI in their development processes. For example, Alphabet’s investor relations report for the 2024 Q3 earnings call states that more than a quarter of new code produced at Google is generated with the assistance of AI systems and then reviewed by engineers. This demonstrates how AI is increasingly embedded in real-world software development pipelines [3].

The situation is further complicated by a shortage of developers who understand legacy programming languages. For example, the average COBOL programmer is approximately 55 years old, and about 10% of that workforce retires each year. As a result, organizations face increasing pressure to modernize and maintain critical legacy systems while dealing with a shrinking pool of experienced experts [4].

Nowadays, there exist numerous solutions for legacy code refactoring. Current automated refactoring tools, such as SonarQube and standard IDE utilities, primarily rely on deterministic, rule-based mechanisms. In addition, several research frameworks—such as CodART, MANTRA, and RefactorGPT have explored multi-agent or AI-assisted approaches for automating refactorings. While these tools and frameworks demonstrate improvements in specific tasks, they suffer from several critical limitations that hinder their ability to perform systematic, strategic, and architecture-aware refactoring across large codebases.

This research proposes the refactoring decision and planning agent a novel agent designed to serve as the brain of a multi-agent refactoring system Unlike existing frameworks that focus on executing refactorings on individual code elements, the RDP Agent specializes in global, strategic planning across the entire codebase. The RDP Agent acts as an intelligent intermediary between a Code Understanding Agent which analyzes the codebase and produces structured quality reports and a Safe Code Transformation Agent which executes refactorings. Its primary function is to transform a list of detected code smells, quality issues, and architectural anti-patterns into a structured, prioritized, and explainable refactoring plan. We have to input a quality report coming from the first agent called understanding agent it can be a json file containing code smells, metrics, dependencies, class relationships. This agent will output Structured refactoring plan in json format

This project contributes to the United Nations Sustainable Development Goal (SDG) 9: Industry, Innovation, and Infrastructure. Target 9.1 highlights the importance of developing reliable, sustainable, and resilient infrastructure. By providing a tool to modernize legacy software systems, this research supports the creation of stronger digital infrastructure while reducing long-term costs and risks in critical sectors such as finance, healthcare, and government.

Additionally, this project aligns with the Software Engineering research cluster. It addresses core software engineering challenges, including software maintenance and evolution, technical debt management, code quality assurance, and the application of intelligent agent-based systems to automate and enhance classical engineering practices.

1.2 Literature Review

Software maintenance and code quality improvement have become critical areas of research in modern software engineering. With the increasing adoption of AI-assisted development tools such as GitHub Copilot and ChatGPT, developers are able to generate large amounts of code rapidly. While these tools significantly improve development speed and productivity, several studies have raised concerns regarding the quality, maintainability, and security of AI-generated code. As a result, researchers have increasingly focused on techniques for

detecting code smells, managing technical debt, and automating refactoring processes to improve overall software quality.

The field of automated software refactoring has been extensively studied in both academia and industry. Researchers have proposed various approaches ranging from traditional rule-based refactoring techniques and search-based optimization methods to modern artificial intelligence and large language model (LLM)-based frameworks. These approaches aim to identify code smells, improve maintainability, and support developers in applying refactoring operations more efficiently. Table summarizes key studies and tools in automated software refactoring, highlighting their main contributions and supporting evidence.

Table 1 Summary of Related Work in Automated Software Refactoring

Reference	Summary	Evidence / Source
Fowler, M. (2018). <i>Refactoring: Improving the Design of Existing Code (2nd ed.)</i> . Addison-Wesley.	Provides a comprehensive catalog of more than 40 refactoring techniques and introduces the concept of code smells. This book is widely considered a foundational resource for modern refactoring tools and practices.	Addison-Wesley Professional
Xu, Y. et al. (2025). <i>MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration</i> . ACM Conference.	Proposes a multi-agent LLM framework that combines retrieval-augmented generation and collaborative agents to improve automated refactoring. Achieves an 82.8% success rate (582/703 instances), outperforming existing approaches such as RawGPT and EM-Assist.	arXiv:2503.14340
Oueslati, K. et al. (2025). <i>RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring</i> .	Introduces a multi-agent architecture consisting of Planner, Generator, Compiler, and Tester agents. Achieves a 90% median test pass rate and reduces code smells by 52.5%.	arXiv:2511.03153
Karabiyik, M.A. (2025). <i>RefactorGPT: A ChatGPT-based multi-agent framework for automated code refactoring</i> .	Presents a sequential four-agent pipeline capable of automated refactoring across multiple programming languages and reports a 100% error recovery rate across nine refactoring types.	DOI: 10.7717/peerj-cs.3257
Zakeri, M. (2021). <i>CodART: Source Code Automated Refactoring Toolkit</i> .	A search-based refactoring toolkit supporting more than 47 refactoring operations and using ANTLR-based AST transformations with multi-objective optimization.	GitHub: m-zakeri/CodART
Pomian, D. et al. (2024). <i>EM-Assist: Safe Automated Extract Method Refactoring with LLMs</i> .	An IntelliJ plugin integrating LLM capabilities with IDE refactoring tools. Achieves 53.4% recall among top-5 recommendations.	DOI: 10.1145/3663529.3663803

Mkaouer, M.W. et al. (2016). <i>On the use of many quality attributes for software refactoring.</i>	Introduces a many-objective optimization approach (NSGA-III) for refactoring considering multiple software quality attributes.	DOI: 10.1007/s10664-015-9414-4
Tsantalis, N. et al. (2018). <i>Accurate and efficient refactoring detection in commit history.</i>	Introduces RefactoringMiner, a tool that detects refactoring operations in Git repositories with high precision (98%) and recall (87%).	DOI: 10.1145/3180155.3180206
DePalma, K. et al. (2024). <i>Exploring ChatGPT's code refactoring capabilities.</i>	Empirical evaluation showing improvements in 39 out of 40 cases while highlighting limitations of LLM-based refactoring.	DOI: 10.1016/j.eswa.2024.123602
Liu, B. et al. (2025). <i>Exploring the potential of general-purpose LLMs in automated software refactoring.</i>	Large-scale empirical study showing that general-purpose LLMs struggle to identify refactoring opportunities without structured planning mechanisms.	DOI: 10.1007/s10515-025-00484-5

One of the earliest refactoring approaches is traditional rule-based refactoring systems, which were widely researched between 2000 and 2010. These systems use predefined heuristics and static analysis rules to detect code smells and suggest suitable refactoring operations. A well-known example is JDeodorant, an Eclipse plug-in designed to automatically identify common code smells such as Feature Envy, Long Method, God Class, Duplicated Code, and Type Checking. After detection, the tool recommends appropriate refactoring actions like Move Method, Extract Method, or Extract Class to address these problems [5].

Rule-based tools like JDeodorant and the built-in refactoring capabilities of modern IDEs (e.g., Eclipse and IntelliJ) became widely adopted because they provide fast, predictable, and behavior-preserving transformations. These systems are particularly effective for simple refactoring tasks where the design problem can be clearly identified using static analysis rules. However, their functionality is inherently limited because the refactoring logic is defined by manually designed rules. Consequently, they lack deeper understanding of software architecture and cannot effectively handle large, complex systems where multiple design issues interact with each other.

To overcome the limitations of rule-based techniques, researchers introduced search-based software engineering approaches, which became prominent between approximately 2015 and 2020. In these approaches, refactoring is formulated as an optimization problem where algorithms search for the best sequence of transformations that improve software quality metrics. For example, the CodART (Source Code Automated Refactoring Toolkit) applies automated transformations on program syntax trees and can generate sequences of refactorings aimed at improving code quality [6].

Similarly, research by Mkaouer and colleagues proposed modeling refactoring as a many-objective optimization problem using evolutionary algorithms such as NSGA-III. In this approach, refactoring solutions are evaluated across many quality metrics simultaneously (sometimes more than fifteen objectives), including coupling, cohesion, complexity, and maintainability. Experimental results showed that such optimization techniques could correct a large percentage of code smells in real-world software systems [7].

Search-based refactoring methods represent an important advancement because they are capable of exploring large solution spaces and generating coordinated sequences of transformations rather than isolated fixes. However, these approaches also present several practical challenges. The optimization process can be computationally expensive and may require long execution times when applied to large projects. Furthermore, these techniques rely heavily on software metrics and often lack semantic understanding of program behavior or developer intent. As a result, the generated refactoring solutions may improve numerical quality metrics while producing code structures that are less intuitive for human developers.

With the rapid advancement of artificial intelligence, recent studies have started investigating refactoring approaches based on Large Language Models (LLMs). These methods utilize the contextual understanding ability of large pretrained models to examine and modify source code. Early work in this area often used a single LLM agent, where the system receives the source code along with a prompt such as “refactor this code.” The model then produces an updated version of the code aimed at improving code quality or addressing design issues.

Single-agent LLM systems provide several advantages compared with earlier techniques. Because they are trained on large code repositories, they are capable of understanding programming language syntax, design patterns, and semantic relationships within code. This allows them to produce more human-readable refactorings and support multiple programming languages. However, empirical studies show that these approaches often suffer from inconsistent results and limited reliability. LLM outputs may fail compilation, introduce behavioral changes, or produce different results for the same prompt due to the probabilistic nature of generative models.

To overcome these limitations, recent studies have shifted toward multi-agent LLM frameworks, which represent a modern direction in automated refactoring research. Rather than depending on a single model, these frameworks break the refactoring workflow into several specialized agents that cooperate to analyze code, generate transformations, validate results, and enhance overall quality. For instance, the RefAgent framework proposes a multi-agent architecture for end-to-end software refactoring, where agents interact with development tools and collaboratively examine code and apply transformations [8].

Another recent framework, RefactorGPT, adopts a sequential pipeline of specialized agents including Analyzer, Refactor, Refine, and Fixer components. These agents work together to examine source code, perform structural improvements, and handle potential errors that may occur during the refactoring process [9].

These multi-agent systems significantly improve reliability compared with single-agent approaches because they incorporate validation steps such as compilation checks, testing, and iterative refinement. Nevertheless, many existing frameworks still focus primarily on code generation and validation rather than strategic decision-making. In many cases, planning logic is embedded implicitly within prompts rather than supported by explicit decision-making models or structured evaluation frameworks.

A further direction that remains underexplored is the use of data-driven learning within the refactoring planning stage itself. While recent multi-agent frameworks improve execution reliability through validation and iterative correction, they still rely mainly on prompts, predefined heuristics, or metric-based optimization when selecting refactoring strategies. As a result, they do not fully exploit historical refactoring patterns or prior outcomes to learn

which refactoring actions are most suitable in a given structural context. This suggests an opportunity for hybrid planning approaches that combine explicit architectural reasoning with learned prediction models for candidate suitability, expected quality improvement, and behavioral risk.

In addition to academic research, several industrial tools and services are commonly used to support software quality improvement. Static analysis platforms such as SonarQube, PMD, and similar tools detect bugs, vulnerabilities, and code smells across large codebases and provide metrics related to technical debt [10] [11]. These tools are widely used in continuous integration pipelines to monitor code quality. However, they primarily function as issue detection systems, generating reports rather than automated improvement strategies.

In industry practice, several software quality tools are widely used to assist developers in identifying design problems. Static analysis platforms such as SonarQube and PMD automatically detect bugs, vulnerabilities, and code smells in large codebases. Integrated development environments such as IntelliJ IDEA and Eclipse also provide built-in refactoring operations including Rename Method, Extract Method, and Move Class [12]. However, these tools typically provide isolated recommendations rather than generating structured refactoring plans for large systems.

Overall, although existing research and tools provide valuable capabilities for identifying design problems and applying refactoring transformations, several limitations remain. Many systems provide only isolated recommendations rather than producing a coherent refactoring roadmap for large systems. Optimization-based approaches often lack explainability, while AI-based approaches may suffer from inconsistency and reliability issues. Furthermore, most existing frameworks do not explicitly evaluate alternative refactoring strategies based on criteria such as implementation complexity, risk of breaking functionality, or expected improvement in software quality.

1.3 Research Gap

Automated code refactoring has been widely studied through both rule-based refactoring tools and recent large language model (LLM)-driven systems. Traditional refactoring engines such as CodART primarily focus on applying predefined refactoring operations through syntax-tree transformations and search-based optimization techniques to improve software quality metrics [6]. However, these systems rely heavily on predefined rules or evolutionary algorithms to determine the sequence of refactoring operations, which limits their adaptability and reasoning capability when handling complex real-world software systems. As a result, they struggle to dynamically analyze contextual dependencies and make intelligent decisions about optimal refactoring strategies.

More recently, LLM-based systems have introduced multi-agent architectures to automate different stages of the refactoring process. For example, RefactorGPT decomposes the refactoring workflow into specialized agents responsible for analysis, transformation, refinement, and error correction [9]. This modular architecture improves interpretability and enables iterative improvement of generated code. However, despite this architectural advancement, these systems mainly focus on executing and refining refactoring actions rather than systematically planning an optimal sequence of refactorings before execution. The agents typically rely on prompt-based decisions and sequential transformations instead of

using a structured planning mechanism capable of reasoning about dependencies, execution constraints, or long-term quality impact.

Another limitation observed in existing approaches is the lack of machine-executable refactoring plans. Many systems generate refactoring instructions in natural language or loosely structured outputs that are interpreted by subsequent agents or tools [9]. While this approach works when downstream components are also LLM-based, it introduces ambiguity when deterministic transformations such as AST-based code modification are required. Without precise structural references, instructions such as selecting a specific method or code fragment may become unreliable or require additional manual interpretation, reducing the reliability of automated refactoring pipelines.

Furthermore, existing research typically produces a single refactoring strategy for a given code base. In practice, software refactoring often involves multiple trade-offs, such as minimizing transformation risk, maximizing structural improvement, or maintaining execution stability. Current approaches rarely support generating alternative refactoring plans optimized for different objectives, nor do they incorporate developer preferences or contextual constraints into the planning process.

Another significant limitation is the absence of adaptive planning mechanisms that respond to execution feedback. Although some systems include validation or error-recovery agents to fix incorrect transformations, the initial refactoring plan itself is rarely updated when failures occur. Consequently, the pipeline relies on post-hoc corrections rather than proactively adjusting the remaining refactoring steps based on runtime feedback or validation outcomes.

Therefore, despite recent progress in LLM-based automated refactoring frameworks, a clear research gap exists in intelligent refactoring planning. Current systems primarily focus on executing refactoring operations or correcting errors after they occur, but they lack a structured mechanism for generating executable, dependency-aware, and adaptable refactoring plans that guide the entire transformation process.

In addition to the absence of structured planning, current automated refactoring approaches also show a lack of data-driven decision support at the planning level. Existing rule-based and optimization-based techniques usually depend on fixed heuristics or handcrafted scoring functions, while recent LLM-based systems embed planning decisions implicitly within prompts. Consequently, these approaches do not learn from historical refactoring outcomes or developer feedback to predict which candidate refactoring is most appropriate for a specific code context. This creates a further research gap in hybrid refactoring planning, where machine learning can be used to estimate refactoring suitability, expected quality gain, and behavioral risk while preserving the explainability and dependency-awareness of an explicit planning framework.

To address this limitation, this research proposes the Refactoring Decision and Planning Agent (RDP Agent) as a dedicated planning component within a multi-agent refactoring architecture. Unlike existing approaches, the proposed agent introduces a planning-centered mechanism that performs structured decision-making before any transformation is applied. The RDP Agent generates machine-executable refactoring plans that explicitly define transformation targets, refactoring parameters, dependencies between operations, and validation checkpoints. This structured plan representation allows downstream transformation

agents to execute refactoring operations more reliably, particularly when working with AST-based code modification techniques.

The novelty of this research lies in integrating structured planning, executable refactoring representations, and adaptive decision-making into a single planning agent. The proposed approach also supports generating multiple candidate refactoring plans optimized for different objectives, such as low-risk modifications, balanced improvements, or deeper structural restructuring. In addition, the planner can update or repair the remaining refactoring steps based on feedback from downstream validation agents, enabling adaptive replanning when failures occur [9].

By introducing a reasoning-driven planning layer before transformation execution, this research extends existing automated refactoring frameworks and bridges the gap between high-level refactoring strategy generation and reliable low-level code transformation. This planning-based approach has the potential to significantly improve the robustness, flexibility, and scalability of automated multi-agent refactoring systems.

2. OBJECTIVE

2.1 Main Objective

The main objective of this research is to design and evaluate a hybrid Refactoring Decision and Planning Agent that integrates rule-based reasoning, machine learning-based suitability prediction, and dependency-aware planning to produce machine-executable refactoring plans. These plans aim to improve the executability, reliability, and overall software quality of automated refactoring compared to existing approaches that lack planning or learning-based decision support.

2.2 Specific Objectives

To achieve the main objective, several specific objectives are defined to guide the design, implementation, and evaluation of the proposed Refactoring Decision and Planning (RDP) Agent. The first objective is to design a structured planning framework capable of analyzing detected code smells and software quality metrics to identify suitable refactoring strategies. This involves detecting potential refactoring opportunities, examining dependencies between refactoring operations, and generating an ordered sequence of actions that improves software maintainability and structural quality.

The second objective is to develop a machine-executable representation of refactoring plans that enables clear communication between the planning agent and downstream transformation agents. The generated plan will include references to target code elements, refactoring types, required parameters, dependencies between operations, and validation checkpoints. This structured format allows the Safe Code Transformation Agent to execute refactoring steps accurately and consistently without ambiguity.

The third objective is to implement a hybrid refactoring decision mechanism that combines machine learning-based suitability prediction with multi-criteria decision analysis to evaluate and rank alternative refactoring strategies. The machine learning component will estimate the expected quality improvement, contextual suitability, and potential behavioral risk of

candidate refactorings using software metrics and structural features, while the decision layer will integrate these predictions with dependency constraints and planning criteria to select the most suitable refactoring sequence.

The fourth objective is to incorporate an adaptive planning mechanism that enables the RDP Agent to update or repair refactoring plans based on feedback from downstream agents such as the validation or execution components. This objective aims to improve the robustness of the refactoring pipeline by allowing the system to dynamically adjust the remaining refactoring steps when failures, conflicts, or unexpected code conditions occur during execution.

The final objective is to evaluate the effectiveness of the proposed planning agent by conducting experimental studies on selected open-source software projects. The evaluation will measure the executability of generated refactoring plans, the success rate of automated refactoring operations, and improvements in software quality metrics including complexity, modularity, and maintainability. The results will be compared with baseline approaches that perform refactoring without an explicit planning mechanism.

3. METHODOLOGY

This research follows a Design Science Research (DSR) methodology together with experimental evaluation to develop and assess the proposed Refactoring Decision and Planning Agent (RDP Agent). The Design Science approach is suitable because the study focuses on creating a new software artifact that addresses the existing research gap in automated refactoring planning. The artifact is designed to produce structured, machine-executable refactoring plans that support automated code transformation workflows.

The proposed system operates within a multi-agent automated refactoring framework, where different agents perform specialized tasks during the refactoring process. Initially, the Code Understanding and Quality Assessment Agent performs static analysis on the source code to identify structural characteristics such as code smells, software metrics, and dependency relationships. These analysis results are provided as structured input to the Refactoring Decision and Planning Agent, which is responsible for generating an ordered and executable refactoring strategy.

The planning process follows a hybrid decision-making approach that combines rule-based reasoning with machine learning-based prediction. First, candidate refactoring operations are generated by mapping detected code smells to appropriate refactoring techniques based on established refactoring knowledge bases. For each candidate refactoring, the system extracts structural features from the code context, including software metrics such as cyclomatic complexity, coupling, cohesion, maintainability index, and dependency relationships between program elements.

These extracted features are provided as input to a machine learning prediction module, which estimates the contextual suitability, expected quality improvement, and potential behavioral risk associated with each candidate refactoring operation. The prediction results are then integrated into a decision-making mechanism that evaluates alternative refactoring strategies. A multi-criteria decision analysis (MCDA) process combines machine learning predictions with additional planning criteria and dependency constraints to determine the most appropriate sequence of refactoring operations.

The proposed system will be implemented using Python, which provides strong support for integrating static analysis tools, multi-agent architectures, and machine learning frameworks. Static code analysis will be performed using Abstract Syntax Tree (AST) parsing tools and software quality analysis frameworks to identify refactoring opportunities within the source code.

For training the machine learning component, datasets will be collected from open-source software repositories. Each data instance will represent a code segment associated with a particular code smell, along with its structural metrics and the corresponding refactoring action. The dataset will be split into training, validation, and testing sets to support model development. Machine learning algorithms implemented through libraries such as Scikit-learn or XGBoost will be used to build predictive models that estimate the suitability and potential impact of candidate refactoring operations.

To assess the effectiveness of the proposed RDP Agent, experimental evaluations will be performed using selected benchmark software projects. The evaluation will examine the capability of the planning agent to produce executable and effective refactoring plans. Key metrics will include Plan Executability Rate, Refactoring Success Rate, and improvements in software quality indicators such as cyclomatic complexity, maintainability index, coupling, cohesion, and code size. The machine learning models will also be evaluated using common predictive performance measures such as accuracy, precision, recall, and F1-score, or regression metrics including Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), depending on the prediction task.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

The proposed automated refactoring framework follows a multi-agent architecture in which different agents collaborate to analyze software systems, generate refactoring strategies, and safely apply transformations. The system is designed to separate analysis, planning, transformation, and validation responsibilities into specialized components to improve modularity and reliability. The overall architecture of the proposed system is illustrated in Figure 1.

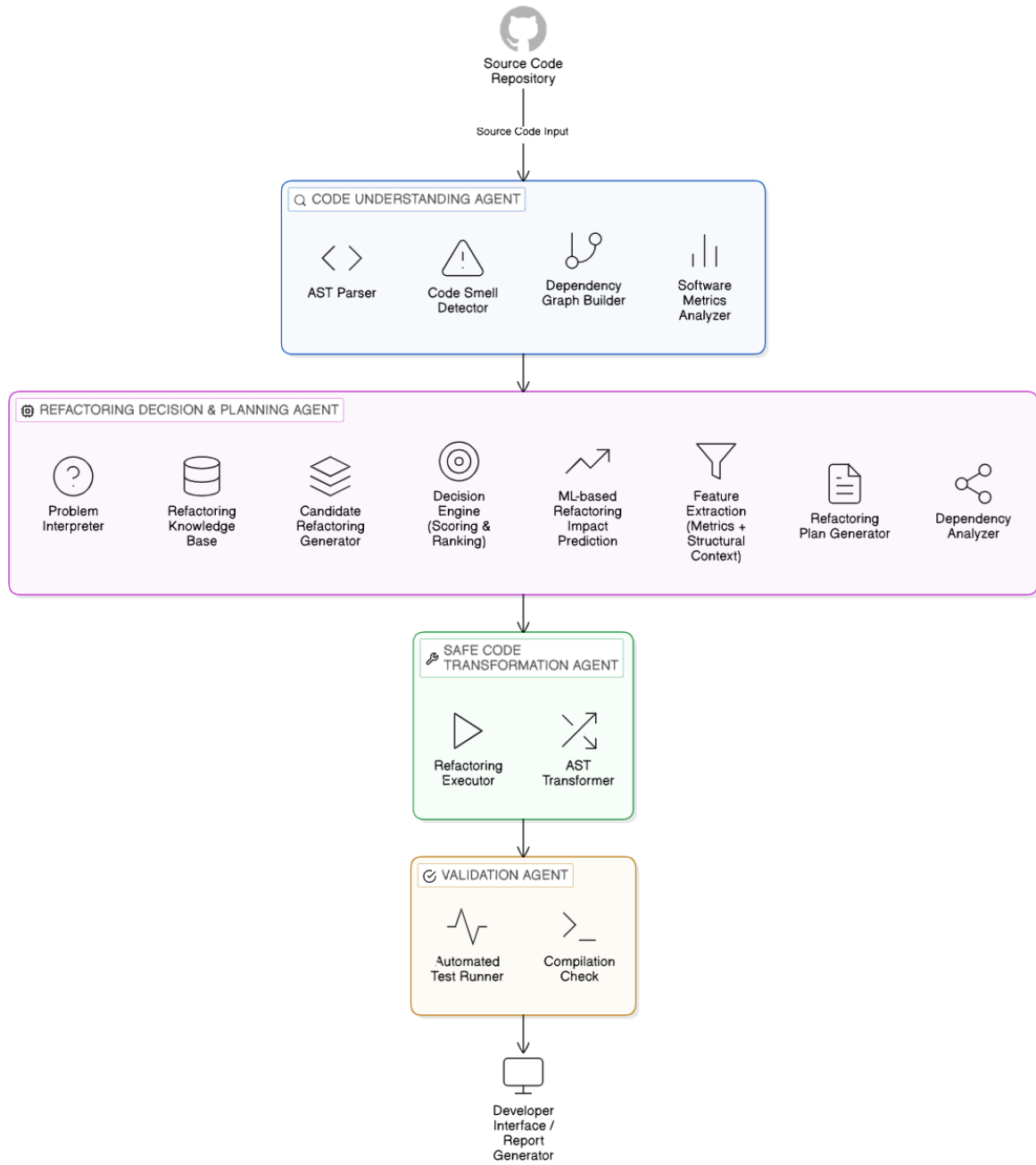


Figure 1 High-Level Architecture of the Automated Multi-Agent Refactoring System

The above diagram shows the overall architecture of the proposed automated refactoring system. The actions in the above system start with the Code Understanding Agent, which analyzes the source code and extracts structural information such as code smells, software metrics, and dependency relationships. These analysis results are then passed to the Refactoring Decision and Planning Agent (RDP Agent), which generates an optimized sequence of refactoring operations. The Safe Code Transformation Agent executes the planned refactorings using AST-based transformations, and the Validation Agent verifies the correctness of the refactored code through compilation and automated testing.

The architecture consists of four main components: the Code Understanding Agent, the Refactoring Decision and Planning Agent (RDP Agent), the Safe Code Transformation Agent, and the Validation Agent. The Code Understanding Agent is responsible for analyzing the source code and extracting structural information such as code smells, software metrics, and dependency graphs. The RDP Agent uses this analysis data to generate structured refactoring plans by evaluating candidate refactoring strategies and determining an optimal execution sequence. The Safe Code Transformation Agent performs the actual code modifications using abstract syntax tree (AST) transformations, while the Validation Agent ensures that the refactored code preserves functional correctness through compilation and automated testing.

Within the enhanced architecture, the RDP agent will contain a machine learning-based suitability and impact prediction submodule in addition to the existing rule-based and decision-making components. After the Code Understanding Agent produces software metrics, code smells, and dependency information, the RDP agent will first generate valid candidate refactorings using a refactoring knowledge base. The machine learning submodule will then estimate the contextual suitability, expected quality improvement, and behavioral risk of each candidate. These predictions will be passed to the decision engine, where they will be combined with dependency analysis and multi-criteria planning rules to produce the final ordered refactoring plan. The Safe Code Transformation Agent will remain deterministic and responsible only for executing the selected plan using AST-based transformations.

The system components interact sequentially through clearly defined data exchanges. The Code Understanding Agent produces analysis results that are forwarded to the RDP Agent for decision making and planning. The generated refactoring plan is then passed to the Safe Code Transformation Agent, which applies the required transformations to the source code. After transformation, the modified code is forwarded to the Validation Agent to ensure that the system behavior remains unchanged. This modular interaction between agents allows each component to specialize in a specific task while maintaining a clear and structured processing pipeline.

The data flow within the system begins with the input source code repository, which is analyzed by the Code Understanding Agent. This agent extracts relevant information including detected code smells, software metrics, and structural dependencies. These analysis results are transmitted to the Refactoring Decision and Planning Agent, which generates an optimized sequence of refactoring operations. The generated plan is then executed by the Safe Code Transformation Agent, which performs AST-based code modifications. Finally, the Validation Agent evaluates the transformed code by performing compilation checks and automated tests before presenting the results to the developer.

The Refactoring Decision and Planning Agent (RDP Agent) is the core component responsible for generating structured refactoring strategies based on the results of code analysis. The agent interprets detected code smells and software metrics, generates candidate refactoring options, evaluates their potential impact on software quality, and determines an optimal sequence of refactoring operations while respecting dependency constraints. The internal architecture of the RDP Agent is shown in Figure 2.

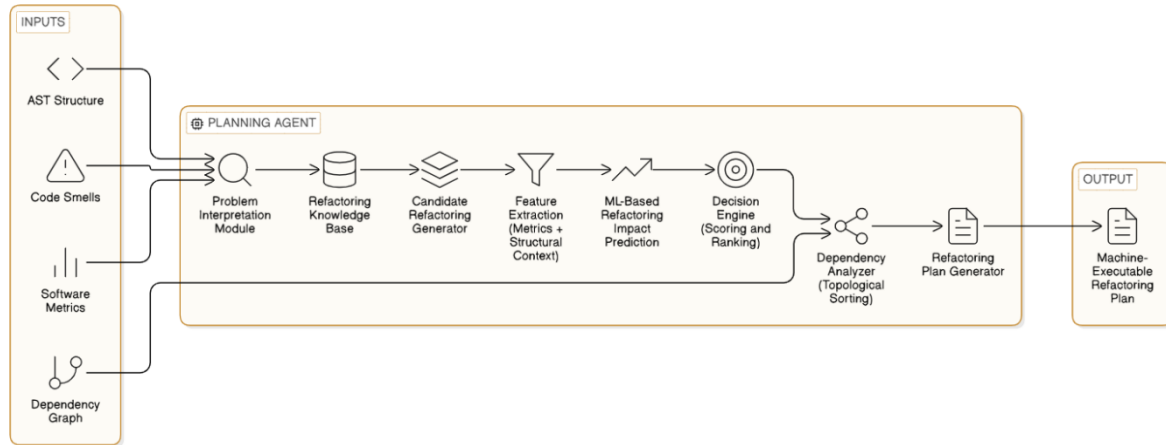


Figure 2 Internal Architecture of the Refactoring Decision and Planning Agent

The above diagram illustrates the internal modules of the Refactoring Decision and Planning Agent. The planning process begins with the Problem Interpretation Module, which analyzes code smells, software metrics, and structural information obtained from the analysis phase. The Refactoring Knowledge Base provides information about available refactoring techniques and their applicability. The Candidate Refactoring Generator produces possible refactoring alternatives, while the Refactoring Impact Prediction Module evaluates their potential effects on software quality attributes such as complexity and coupling. The Decision Engine ranks these alternatives based on defined criteria, and the Dependency Analyzer determines a valid execution order for the selected operations. Finally, the Refactoring Plan Generator produces a machine-executable refactoring plan for the transformation agent.

The planning process performed by the RDP Agent involves multiple stages that transform raw analysis results into an executable refactoring strategy. This process includes identifying refactoring opportunities, generating candidate solutions, predicting the impact of each transformation, and determining an optimal sequence of operations. The workflow of this planning actions are showed in Figure 3

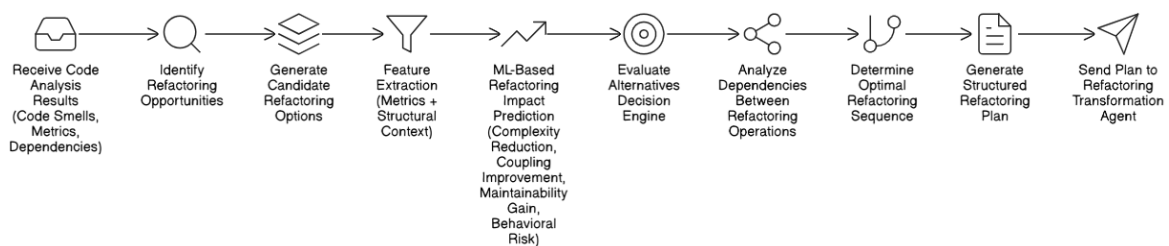


Figure 3 Workflow of the Refactoring Planning Process

The workflow of the Refactoring Decision and Planning Agent describes the process through which raw code analysis results are transformed into a structured and machine-executable refactoring plan. The process begins when the RDP Agent receives a structured quality report

generated by the Code Understanding and Quality Assessment Agent, which includes detected code smells, software quality metrics, and dependency information extracted from the source code. Based on this information, the planning agent interprets the detected issues and identifies potential refactoring opportunities within the affected code regions.

After identifying the issues, the agent generates multiple candidate refactoring operations that could potentially improve the software structure. For each candidate refactoring, the system extracts a set of structural features from the code context, including software metrics such as cyclomatic complexity, coupling between classes, cohesion indicators, method size, and dependency relationships. These extracted features are provided as input to a machine learning-based prediction module that estimates the contextual suitability, expected quality improvement, and potential behavioral risk associated with each candidate refactoring operation.

The predictions produced by the machine learning model are then integrated into a decision-making process that evaluates alternative refactoring strategies. A multi-criteria decision analysis (MCDA) mechanism combines machine learning predictions with additional criteria such as implementation complexity, architectural impact, and dependency constraints. Based on this evaluation, the planning agent ranks the candidate refactoring options and selects the most appropriate transformations.

Once the best refactoring candidates are selected, the system analyzes dependencies between the chosen refactoring operations to determine a valid execution order. Dependency relationships are used to construct an ordered sequence of refactoring actions that avoids conflicts and preserves program correctness. Finally, the Refactoring Plan Generator produces a structured and machine-readable refactoring plan that specifies the selected transformations, their execution order, required parameters, and validation checkpoints. This machine-executable plan is then provided to the Safe Code Transformation Agent for controlled application to the source code.

USER REQUIREMENTS

5.1 Stakeholder Identification

The proposed automated refactoring system involves several stakeholders who interact with or benefit from the system. The primary stakeholders are software developers, who use the system to automatically identify code smells and apply safe refactoring operations to improve software maintainability and code quality. By providing automated refactoring planning and execution, the system helps developers reduce manual effort and minimize errors when restructuring complex codebases.

Another important stakeholder group is software development teams and project maintainers, who are responsible for maintaining large and evolving code repositories. The proposed system can assist these stakeholders by generating structured refactoring plans that improve code modularity, reduce technical debt, and enhance long-term maintainability of software systems.

Software engineering researchers and students are also potential stakeholders, as the system can serve as an experimental platform for studying automated refactoring techniques, multi-agent architectures, and intelligent planning mechanisms in software engineering research.

Finally, software organizations and technology companies can benefit from the system by integrating automated refactoring capabilities into development workflows to improve software quality, reduce maintenance costs, and increase development productivity.

5.2 Functional Requirements

The below table explains the core capabilities that the proposed automated refactoring system planning agent must provide.

Table 2 Functional Requirements of the Refactoring Decision and Planning Agent

ID	Functional Requirement Description
FR1	The Refactoring Decision and Planning Agent shall receive structured analysis results, including detected code smells, software quality metrics, and dependency information, from the Code Understanding Agent.
FR2	The RDP Agent shall analyze the received results to detect potential refactoring opportunities within the source code.
FR3	The RDP Agent shall map identified code smells and quality issues to suitable refactoring techniques using a refactoring knowledge base derived from established refactoring catalogs.
FR4	The RDP agent shall generate multiple candidate refactoring alternatives for the identified code regions to allow evaluation of different improvement strategies.
FR5	The RDP agent shall evaluate candidate refactoring alternatives using decision-making techniques such as weighted scoring or optimization methods to determine the most beneficial transformation.
FR6	The RDP agent shall determine an optimal sequence of refactoring operations by considering dependencies between refactoring actions and structural constraints in the codebase.
FR7	The RDP agent shall produce a structured and machine-readable refactoring plan (e.g., JSON format) that can be executed by the Safe Code Transformation Agent.
FR8	The RDP agent shall extract structured features from code smells, software metrics, and dependency graphs to serve as input features for machine learning models.
FR9	The RDP agent shall utilize a trained machine learning model to predict contextual suitability, expected quality improvement, and behavioral risk of candidate refactorings.
FR10	The RDP agent shall incorporate machine learning prediction outputs into the candidate ranking and sequencing process together with dependency-aware planning constraints.
FR11	The RDP agent shall store validation outcomes and optional developer feedback to support future model retraining, calibration, or analytical evaluation.

5.3 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational constraints of the proposed planning agent.

Table 3 Non-Functional Requirements of the Refactoring Decision and Planning Agent

Category	Description
Performance	The Refactoring Decision and Planning Agent shall generate refactoring plans within a reasonable execution time to support automated refactoring workflows.
Scalability	The RDP agent shall support planning for large-scale software projects by efficiently processing analysis results generated from large codebases.
Reliability	The RDP agent shall generate consistent and conflict-free refactoring plans that minimize the risk of incorrect, unsafe, or incompatible refactoring actions.
Maintainability	The architecture of the RDP agent shall be modular to allow easy modification, debugging, or improvement of planning strategies and system components.
Extensibility	The RDP agent shall support the integration of new refactoring rules, decision algorithms, or planning strategies without requiring significant architectural redesign.
Explainability	The RDP agent shall provide interpretable reasoning explaining why a candidate refactoring receives a high or low suitability score.
Reproducibility	Given the same input analysis report and model version, the RDP agent shall produce consistent candidate scores and refactoring rankings.
Model Efficiency	The machine learning prediction stage shall operate within acceptable latency to ensure that the overall planning workflow remains practical for automated refactoring pipelines.

COMMERCIALIZATION PLAN

In this system has strong commercialization potential within the global software development and software maintenance industry. The primary target market includes software development companies, technology startups, and large enterprises that maintain complex software systems or legacy codebases. These organizations frequently struggle with technical debt, poor code quality, and the increasing complexity of maintaining large software systems over time. Development teams that actively use AI-assisted coding tools such as GitHub Copilot or other large language model-based coding assistants represent a particularly relevant customer segment, as these tools often accelerate code generation without ensuring long-term architectural quality. The main users of the system would include software developers, software architects, DevOps engineers, and quality assurance teams responsible for maintaining and improving code quality.

The value proposition of the proposed system lies in its ability to provide an automated and intelligent refactoring workflow that analyzes code quality issues and generates structured refactoring plans before performing code transformations. Unlike traditional static analysis tools that only detect code smells or provide isolated recommendations, the proposed multi-agent framework integrates code analysis, strategic refactoring planning, safe code transformation, and validation into a unified pipeline. The Refactoring Decision and Planning Agent (RDP Agent) plays a key role in this architecture by generating dependency-aware refactoring plans that guide the transformation process and ensure that refactoring actions are applied in an optimal sequence.

From a commercialization perspective, the system can be delivered as a developer productivity platform integrated into modern software development environments. The system could be deployed as an integrated development environment (IDE) plugin, a cloud-based code quality analysis service, or an enterprise software maintenance platform integrated with version control systems such as GitHub or GitLab. A Software-as-a-Service (SaaS) revenue model could be adopted, where development teams subscribe to the platform on a monthly or annual basis to analyze repositories, generate refactoring plans, and improve software quality across projects.

The operational costs associated with commercial deployment would primarily include cloud infrastructure for code analysis and machine learning processing, platform maintenance and updates, and technical support services for enterprise users. A tiered pricing strategy could be implemented to accommodate different types of customers. For example, smaller development teams and startups could access a basic version of the platform with limited analysis capabilities, while larger enterprises could purchase premium plans that support large-scale repository analysis, advanced refactoring planning, integration with DevOps pipelines, and team collaboration features.

The system maintains a competitive advantage by providing a planning-driven automated refactoring solution that considers dependencies, architectural constraints, and expected quality improvements before executing transformations. While many existing tools focus only on detecting code smells or performing individual refactoring actions, the proposed framework introduces an intelligent planning layer that coordinates multiple refactoring operations within a structured workflow. This approach enables safer and more effective large-scale refactoring of complex software systems.

Regarding intellectual property, the key innovation of the system is its multi-agent architecture and the planning mechanisms designed to produce structured refactoring strategies. The algorithms, decision models, and planning framework developed in this research can be protected through software copyright or intellectual property registration. Furthermore, proprietary machine learning models and training datasets used to predict refactoring suitability may offer additional competitive benefits for potential commercial deployment of the system.

7. BUDGET AND JUSTIFICATION

The proposed research mainly requires cloud computing resources to run experiments, store datasets, and host the prototype system. Most software tools used in the project such as Python libraries, SonarQube Community Edition, and open-source frameworks are freely available. Therefore, the required budget mainly focuses on minimal cloud infrastructure and hosting services.

Table 4 Estimated Budget for System Implementation and Experimentation

Resource	Purpose	Estimated Cost (LKR)
Cloud Compute Instance (AWS EC2 / DigitalOcean Droplet)	Running code analysis agents and refactoring planning experiments	LKR 7,798.75 / month
Cloud Storage (AWS S3 / Google Cloud Storage)	Storing datasets, repositories, and generated refactoring plans	LKR 935.85 / month

Small Database Service (AWS RDS / Managed PostgreSQL)	Storing software metrics, code quality reports, and experiment results	LKR 3,119.50 / month
Web Hosting / Server Hosting	Hosting the prototype system interface and API	LKR 3,119.50 / month
Domain and Backup Storage	Domain registration and backup of experiment data	LKR 623.90 / month

Total Estimated Budget

Total Cost: 15,597.50 LKR per month

Cloud Compute Instance

A cloud server will be used to run the multi-agent refactoring system, perform static code analysis, and execute experiments on open-source software repositories.

Cloud Storage

Cloud storage services will store datasets, software repositories, experiment results, and generated refactoring plans produced during evaluation.

Database Service

A lightweight database will store structured data such as software metrics, dependency relationships, and refactoring planning outputs.

Web Hosting

A basic hosting service will deploy the prototype system interface, allowing users to submit code quality reports and obtain refactoring plans generated by the planning agent.

Domain and Backup Storage

A small budget is allocated for domain registration and backup storage to ensure reliability and accessibility of the experimental system.

8. WORK BREAKDOWN STRUCTURE (WBS)

The project work is divided among the four team members according to the specialized agents in the multi-agent architecture. Each member is responsible for the design, implementation, and evaluation of a specific agent while collaborating on common tasks such as system integration, testing, and documentation.

Table 5 Task Distribution Among Team Members

Team Member	Agent	Key Responsibilities
Member 1	Code Understanding Agent	Analyzing the source code structure, identifying code smells, generating complexity metrics, and producing structured quality reports.
Member 2	Refactoring Planning Agent	Generating intelligent refactoring strategies based on analysis results, prioritizing refactoring actions, and preparing machine-executable refactoring plans.
Member 3	Safe Code Transformation & Validation Agent	Applying AST-based code transformations, validating syntactic and semantic correctness, performing behavioral validation, and handling rollback when necessary.
Member 4	Developer Interaction & Workflow Orchestration Agent	Managing the interaction between developers and the system, coordinating workflow among agents, presenting refactoring plans and results through a dashboard or IDE interface.

Although each member is responsible for a specific component, several tasks are performed collaboratively to ensure system consistency and integration. These shared tasks include system architecture design, integration of all agents into a unified platform, testing and validation of the complete system, evaluation of experimental results, and preparation of research documentation.

The major development activities of the project are organized into phases that align with the research timeline. The project begins with literature review and system design activities, followed by implementation of the individual agents. After the development stage, the agents are integrated into a unified system and evaluated using selected legacy code repositories. Finally, documentation and final reporting activities are completed.

Table 6 Project Timeline Alignment

Project Phase	Activities	Timeline
Phase 1	Literature review, problem analysis, and research proposal preparation	Month 1 – Month 2
Phase 2	System architecture design and agent specification	Month 3
Phase 3	Implementation of Code Understanding Agent	Month 4 – Month 5
Phase 4	Implementation of Refactoring Planning Agent	Month 4 – Month 5
Phase 5	Implementation of Safe Transformation & Validation Agent	Month 5 – Month 6

Phase 6	Implementation of Developer Interaction & Workflow Orchestration Agent	Month 5 – Month 6
Phase 7	Integration of agents and system testing	Month 7
Phase 8	Experimental evaluation and performance analysis	Month 8
Phase 9	Final documentation, report writing, and presentation preparation	Month 9

The workload distribution among team members is carefully balanced so that each member contributes equally to the development of the proposed system. Each agent requires comparable levels of research, design, and implementation effort. In addition, shared activities such as system integration, testing, evaluation, and documentation are carried out collaboratively by all members. This balanced distribution of tasks ensures efficient progress of the project while allowing each team member to develop expertise in a specific component of the multi-agent architecture.

9. GANTT CHART

The project is organized into multiple phases including literature review, system design, implementation of individual agents, system integration, evaluation, and final documentation. The schedule ensures that each stage of the project is completed within the academic timeline while allowing sufficient time for development, testing, and analysis.

Table 7 Gantt chart

Project Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6	Month 7	Month 8	Month 9
Literature Review	●	●							
Problem Identification	●	●							
System Architecture Design		●	●						
Research Proposal Preparation	●	●							
Development of Code Understanding Agent			●	●					
Development of Refactoring Planning Agent			●	●					
Development of Transformation & Validation Agent				●	●				
Development of Workflow Orchestration Agent				●	●				

System Integration					•	•			
Testing and Validation						•	•		
Experimental Evaluation							•	•	
Final Report Writing							•	•	•
Final Presentation Preparation								•	•

10.REFERENCES LIST

[1]	C. f. I. & S. Q. (CISQ), "The Cost of Poor Software Quality in the US: A 2022 Report," IT-CISQ, 2022. [Online]. Available: https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2022-report/ .
[2]	K. Studios, "The State of Vibe Coding: A 2026 Strategic Blueprint," Keywords Studios, 2026. [Online]. Available: https://www.keywordsstudios.com/en/about-us/news-events/news/the-state-of-vibe-coding-a-2026-strategic-blueprint/ .
[3]	A. Inc., "Alphabet Q3 2024 Earnings Call," Alphabet Investor Relations, 2024. [Online]. Available: https://abc.xyz/investor/events/event-details/2024/2024-q3-earnings-call/ .
[4]	AdwaitX, "AI COBOL Modernization and the Cost Barrier," AdwaitX, 2026. [Online]. Available: https://www.adwaitx.com/ai-cobol-modernization-cost-barrier-2026/ .
[5]	N. Tsantalis, "JDeodorant: A Tool for Detecting Code Smells," GitHub, 2023. [Online]. Available: https://github.com/tsantalis/JDeodorant . [Accessed 5 March 2026].
[6]	M. Zakeri, "CodART: Source Code Automated Refactoring Toolkit," CodART Project, 2021. [Online]. Available: https://www.m-zakeri.ir/codart-automated-source-code-refactoring-toolkit.html . [Accessed 5 3 2026].
[7]	M. W. Mkaouer, "Publications – Mohamed Wiem Mkaouer," Rochester Institute of Technology, 2024. [Online]. Available: https://www.se.rit.edu/~mwm/publication.html . [Accessed 5 March 2026].
[8]	O. K. L. M. K. and F. , "RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring," arXiv, p. arXiv:2511.03153, 2025.
[9]	M. A. Karabiyik, "RefactorGPT: A ChatGPT-based multi-agent framework for automated code refactoring," PeerJ Computer Science, vol. 11, p. e3257, 2025.
[10]	SonarSource, "SonarQube Documentation," SonarSource, 2024. [Online]. Available: https://docs.sonarsource.com/ . [Accessed 5 March 2026].
[11]	P. D. Team, "PMD Source Code Analyzer," PMD, 2024. [Online]. Available: https://pmd.github.io/ . [Accessed 5 March 2026].
[12]	M. Fowler and K. Beck, Refactoring: Improving the Design of Existing Code, 2 ed., Boston: Addison-Wesley Professional, 2018.

11.APPENDICES

Appendix A Detailed Multi-Agent Refactoring System Architecture

The automated refactoring system follows a multi-agent architecture consisting of four specialized agents.

1. Code Understanding & Quality Assessment Agent
2. Refactoring Decision & Planning Agent
3. Safe Code Transformation & Validation Agent
4. Developer Interaction & Workflow Orchestration Agent

The first component of the architecture is the Code Understanding and Quality Assessment Agent, which analyzes legacy source code to extract structural and quality-related information. Instead of treating the source code as plain text, this agent parses the code using Abstract Syntax Tree (AST) analysis, allowing the system to understand program structure at a semantic level. Through this process, the agent identifies classes, methods, inheritance relationships, and dependencies between modules. Additionally, it computes several software quality metrics including cyclomatic complexity, coupling between objects, cohesion among methods, and lines of code. The output of this agent is a structured quality report that summarizes detected code smells, metric values, and dependency relationships.

The second component is the Refactoring Decision and Planning Agent which serves as the central decision-making unit of the system. This agent receives the structured quality report produced by the Code Understanding Agent and evaluates potential refactoring opportunities. Instead of immediately applying code transformations, the agent generates a strategic refactoring roadmap that specifies which elements of the system should be refactored and in what order. The decision-making process relies on a Multi-Criteria Decision Analysis (MCDA) model that evaluates alternative refactoring strategies based on criteria such as expected quality improvement, implementation complexity, and the risk of behavioral changes.

The third component is the Safe Code Transformation and Validation Agent, which is responsible for executing the refactoring plan generated by the agent. Refactoring operations are applied using AST-based transformation mechanisms to ensure structural correctness. After each transformation, the agent performs validation through structural analysis, compilation checks, and optional automated testing to verify that the refactored code preserves the behavior of the original program.

Finally, the system includes a Developer Interaction and Workflow Orchestration Agent, which coordinates the interaction between the different agents and provides a human-in-the-loop interface for developers. Through this interface, developers can review suggested refactoring plans, monitor quality improvements, and approve or reject automated transformations. This hybrid design combines the efficiency of automated analysis with human oversight to ensure reliable software evolution.

Appendix B Internal Architecture of the Refactoring Decision & Planning Agent

The Refactoring Decision and Planning Agent is designed as a modular system that converts software quality analysis results into structured refactoring strategies. The internal architecture of the RDP Agent consists of several interconnected modules responsible for processing input data, identifying refactoring opportunities, evaluating alternative strategies, and generating executable refactoring plans.

The first module is the Input Processing Module, which receives the structured quality reports generated by the Code Understanding Agent. These reports contain detailed information about code smells, software metrics, and dependency relationships between program elements. The module parses this information and converts it into internal data structures suitable for further analysis.

The second module is the Refactoring Opportunity Identification Module, which determines potential refactoring actions by mapping detected code smells to appropriate refactoring techniques. For instance, a long method may be associated with the Extract Method refactoring, while a God Class may indicate the need for Extract Class refactoring. This mapping is based on established refactoring catalogs such as those proposed by Fowler.

The third module is the Strategy Generation Module, which produces multiple candidate refactoring strategies. Each candidate plan consists of a sequence of refactoring actions designed to resolve detected design issues. Because multiple solutions may exist for a given problem, this module generates alternative plans that differ in scope, complexity, and execution order.

The fourth module is the Decision Evaluation Module, which evaluates candidate refactoring strategies using a Multi-Criteria Decision Analysis model. This evaluation considers multiple criteria including expected software quality improvement, implementation complexity, risk of introducing behavioral changes, and the potential impact on system dependencies.

Finally, the Refactoring Plan Generation Module selects the highest-ranked strategy and converts it into a machine-executable refactoring plan. This structured representation includes explicit references to target classes or methods, the type of refactoring to be applied, and the recommended execution order.

Appendix C Refactoring Decision Algorithm

The decision-making process of the Refactoring Decision and Planning Agent follows a structured algorithm that transforms software quality reports into executable refactoring plans.

First, the system receives a code quality report generated by the Code Understanding Agent. This report contains software metrics, detected code smells, and dependency information. The agent then parses this report and extracts relevant structural information about the software system.

Next, the agent identifies potential refactoring opportunities by analyzing detected code smells and mapping them to appropriate refactoring techniques. Once potential improvements

are identified, the system generates several candidate refactoring strategies that attempt to resolve the detected design issues.

Each candidate strategy is evaluated using the Multi-Criteria Decision Analysis model. The evaluation considers multiple criteria including expected improvement in software maintainability, implementation complexity, risk of behavior modification, and dependency conflicts between refactoring operations.

After evaluation, the candidate strategies are ranked according to their computed scores. The system then performs dependency analysis to determine a safe execution order for the selected refactoring operations. Finally, the agent generates a structured refactoring plan and forwards it to the Safe Code Transformation Agent for execution.

Appendix D Example Code Quality Report (Input)

The following example illustrates a simplified code quality report generated by the Code Understanding Agent and used as input for the Refactoring Decision and Planning Agent.

```
{
  "class_name": "OrderService",
  "metrics": {
    "cyclomatic_complexity": 18,
    "coupling_between_objects": 7,
    "lines_of_code": 235
  },
  "code_smells": [
    "Long Method",
    "God Class",
    "High Coupling"
  ],
  "dependencies": [
    "PaymentService",
    "CustomerRepository",
    "InvoiceManager"
  ]
}
```

This report indicates that the OrderService class has relatively high complexity and coupling values. Such conditions may reduce maintainability and increase the risk of defects. Based on this information, the agent identifies possible refactoring opportunities to improve the modularity and readability of the code.

Appendix E Example Refactoring Plan Output

After analyzing the input quality report, the Refactoring Decision and Planning Agent produces a structured refactoring plan that outlines the recommended improvements.

```
{
  "refactoring_plan": [
    {
```

```

    "target_class": "OrderService",

    "target_method": "processPayment",

    "refactoring_type": "Extract Method",

    "priority": 1,

    "predicted_quality_gain": 0.81,

    "predicted_behavioral_risk": 0.14,

    "suitability_score": 0.87,

    "model_confidence": 0.79
  },
  {

    "target_class": "OrderService",

    "refactoring_type": "Extract Class",

    "new_class_name": "PaymentProcessor",

    "priority": 2,

    "predicted_quality_gain": 0.74,

    "predicted_behavioral_risk": 0.21,

    "suitability_score": 0.76,

    "model_confidence": 0.72
  }
]

```

} This structured representation allows downstream transformation agents to apply refactoring operations automatically while preserving traceability and execution order.

Appendix F MCDA Decision Matrix

In the enhanced RDP Agent, the MCDA process will incorporate machine learning predictions as part of candidate evaluation. Rather than replacing the decision model, the learned outputs will be used as additional evidence, such as predicted quality gain, contextual suitability, or behavioral risk, which will then be normalized and combined with implementation complexity and dependency impact. This preserves interpretability while allowing the planning agent to benefit from data-driven decision support.

The evaluates alternative refactoring strategies using a Multi-Criteria Decision Analysis model. This model enables the system to compare different refactoring options by considering multiple decision criteria simultaneously.

The evaluation process considers several criteria including expected quality improvement, implementation complexity, risk of behavioral change, and the potential impact on dependencies. Each criterion is assigned a weight representing its relative importance in the decision-making process.

For example, the weight configuration may be defined as follows:

Quality Improvement – 0.40

Implementation Complexity – 0.25

Risk of Behavioral Change – 0.20

Dependency Impact – 0.15

The final score of each candidate refactoring strategy is computed using a weighted scoring formula that combines these criteria. Strategies with higher scores are prioritized when generating the final refactoring plan.